

Algoritmos_Emilio_Moreno_AG2

September 7, 2026

1 AG2 - Actividad Guiada 2

Nombre: Emilio Moreno Link: URL:https://github.com/Hiper1961/Algoritmos_Actividades_Guiadas-1-2-3.git

```
[ ]: #Librerias
import math
```

1.1 Programación Dinámica. Viaje por el río

- **Definición:** Es posible dividir el problema en subproblemas más pequeños, guardando las soluciones para ser utilizadas más adelante.
- **Características** que permiten identificar problemas aplicables: -Es posible almacenar soluciones de los subproblemas para ser utilizados más adelante -Debe verificar el principio de optimalidad de Bellman: “en una secuencia optima de decisiones, toda sub-secuencia también es óptima” (*) -La necesidad de guardar la información acerca de las soluciones parciales unido a la recursividad provoca la necesidad de preocuparnos por la complejidad espacial (cuantos recursos de espacio usaremos)

1.1.1 Problema

En un río hay n embarcaderos y debemos desplazarnos río abajo desde un embarcadero a otro. Cada embarcadero tiene precios diferentes para ir de un embarcadero a otro situado más abajo. Para ir del embarcadero i al j , puede ocurrir que sea más barato hacer un trasbordo por un embarcadero intermedio k . El problema consiste en determinar la combinación más barata.

- Consideramos una tabla $TARIFAS(i,j)$ para almacenar todos los precios que nos ofrecen los embarcaderos.
- Si no es posible ir desde i a j daremos un valor alto para garantizar que ese trayecto no se va a elegir en la ruta óptima(modelado habitual para restricciones)

```
[ ]: #Viaje por el río - Programación dinámica
#####

TARIFAS = [
[0,5,4,3,float("inf"),999,999], #desde nodo 0 los valores grandes representa
↪que no hay camino hacia ese punto
```

```
[999,0,999,2,3,999,11], #desde nodo 1
[999,999, 0,1,999,4,10], #desde nodo 2
[999,999,999, 0,5,6,9],
[999,999, 999,999,0,999,4],
[999,999, 999,999,999,0,3],
[999,999,999,999,999,999,0]
]

#999 se puede sustituir por float("inf") del modulo math
TARIFAS
```

```
[ ]: [[0, 5, 4, 3, inf, 999, 999],
      [999, 0, 999, 2, 3, 999, 11],
      [999, 999, 0, 1, 999, 4, 10],
      [999, 999, 999, 0, 5, 6, 9],
      [999, 999, 999, 999, 0, 999, 4],
      [999, 999, 999, 999, 999, 0, 3],
      [999, 999, 999, 999, 999, 999, 0]]
```

```
[ ]: #Calculo de la matriz de PRECIOS y RUTAS
# PRECIOS - contiene la matriz del mejor precio para ir de un nodo a otro
# RUTAS - contiene los nodos intermedios para ir de un nodo a otro
#####
def Precios(TARIFAS):
#####
    #Total de Nodos
    N = len(TARIFAS[0])

    #Inicialización de la tabla de precios
    PRECIOS = [ [9999]*N for i in range(N)] #n x n
    RUTA = [ [""]*N for i in range(N)]

    #Se recorren todos los nodos con dos bucles(origen - destino)
    # para ir construyendo la matriz de PRECIOS
    for i in range(N-1):
        for j in range(i+1, N):
            MIN = TARIFAS[i][j]
            RUTA[i][j] = i

            for k in range(i, j):
                if PRECIOS[i][k] + TARIFAS[k][j] < MIN:
                    MIN = min(MIN, PRECIOS[i][k] + TARIFAS[k][j] )
                    RUTA[i][j] = k
            PRECIOS[i][j] = MIN

    return PRECIOS,RUTA
```

```
[ ]: PRECIOS, RUTA = Precios(TARIFAS)
      #print(PRECIOS[0][6])

      print("PRECIOS")
      for i in range(len(TARIFAS)):
          print(PRECIOS[i])

      print("\nRUTA")
      for i in range(len(TARIFAS)):
          print(RUTA[i])
```

PRECIOS

```
[9999, 5, 4, 3, 8, 8, 11]
[9999, 9999, 999, 2, 3, 8, 7]
[9999, 9999, 9999, 1, 6, 4, 7]
[9999, 9999, 9999, 9999, 5, 6, 9]
[9999, 9999, 9999, 9999, 9999, 999, 4]
[9999, 9999, 9999, 9999, 9999, 9999, 3]
[9999, 9999, 9999, 9999, 9999, 9999, 9999]
```

RUTA

```
['', 0, 0, 0, 1, 2, 5]
['', '', 1, 1, 1, 3, 4]
['', '', '', 2, 3, 2, 5]
['', '', '', '', 3, 3, 3]
['', '', '', '', '', 4, 4]
['', '', '', '', '', '', 5]
['', '', '', '', '', '', '']
```

```
[ ]: #Calculo de la ruta usando la matriz RUTA
      def calcular_ruta(RUTA, desde, hasta):
          if desde == RUTA[desde][hasta]:
              #if desde == hasta:
                  #print("Ir a : " + str(desde))
                  return desde
              else:
                  return str(calcular_ruta(RUTA, desde, RUTA[desde][hasta])) + ',' +
                  str(RUTA[desde][hasta])

      print("\nLa ruta es:")
      calcular_ruta(RUTA, 0,6)
```

La ruta es:

```
[ ]: '0,2,5'
```

1.2 Problema de Asignacion de tarea

```
[ ]: #Asignacion de tareas - Ramificación y Poda
#####
#   T A R E A
#   A
#   G
#   E
#   N
#   T
#   E

COSTES=[[11,12,18,40],
        [14,15,13,22],
        [11,17,19,23],
        [17,14,20,28]]
```

```
[ ]: #Calculo del valor de una solucion parcial
def valor(S,COSTES):
    VALOR = 0
    for i in range(len(S)):
        #print(COSTES[S[i]][i])
        VALOR += COSTES[S[i]][i]
    return VALOR

valor((3,2, ),COSTES) # 3 y dos son los indices del agente tarea 0 agente 3
↳ tarea 1 agente 2
```

[]: 34

```
[ ]: #Coste inferior para soluciones parciales
# (1,3,) Se asigna la tarea 1 al agente 0 y la tarea 3 al agente 1

def CI(S,COSTES):
    VALOR = 0
    #Valores establecidos
    for i in range(len(S)):
        VALOR += COSTES[i][S[i]]

    #Estimacion
    for i in range( len(S), len(COSTES) ):
        VALOR += min( [ COSTES[j][i] for j in range(len(S), len(COSTES)) ] )
    return VALOR

def CS(S,COSTES):
    VALOR = 0
    #Valores establecidos
```

```

for i in range(len(S)):
    VALOR += COSTES[i][S[i]]

#Estimacion
for i in range( len(S), len(COSTES) ):
    VALOR += max( [ COSTES[j][i] for j in range(len(S), len(COSTES)) ] )
return VALOR

CI((0,1),COSTES)

```

[]: 68

```

[ ]: #Genera tantos hijos como posibilidades haya para la siguiente elemento de
      ↪ la tupla
      #(0,) -> (0,1), (0,2), (0,3)
def crear_hijos(NODO, N):
    HIJOS = []
    for i in range(N ):
        if i not in NODO:
            HIJOS.append({'s':NODO +(i,) })
    return HIJOS

```

[]: crear_hijos((0,) , 4)

[]: [{'s': (0, 1)}, {'s': (0, 2)}, {'s': (0, 3)}]

```

[ ]: def ramificacion_y_poda(COSTES):
      #Construccion iterativa de soluciones(arbol). En cada etapa asignamos un
      ↪ agente(ramas).
      #Nodos del grafo { s:(1,2),CI:3,CS:5 }
      #print(COSTES)
      DIMENSION = len(COSTES)
      MEJOR_SOLUCION=tuple( i for i in range(len(COSTES)) )
      CotaSup = valor(MEJOR_SOLUCION,COSTES)
      #print("Cota Superior:", CotaSup)

      NODOS=[]
      NODOS.append({'s':(), 'ci':CI((),COSTES) })

      iteracion = 0

      while( len(NODOS) > 0):
          iteracion +=1

          nodo_prometedor = [ min(NODOS, key=lambda x:x['ci']) ][0]['s']
          #print("Nodo prometedor:", nodo_prometedor)

```

```

#Ramificacion
#Se generan los hijos
HIJOS = [ {'s':x['s'], 'ci':CI(x['s'], COSTES) } for x in
↳crear_hijos(nodo_prometedor, DIMENSION) ]

#Revisamos la cota superior y nos quedamos con la mejor solucion si
↳llegamos a una solucion final
NODO_FINAL = [x for x in HIJOS if len(x['s']) == DIMENSION ]
if len(NODO_FINAL ) >0:
    #print("\n*****Soluciones:", [x for x in HIJOS if len(x['s']) ==
↳DIMENSION ] )
    if NODO_FINAL[0]['ci'] < CotaSup:
        CotaSup = NODO_FINAL[0]['ci']
        MEJOR_SOLUCION = NODO_FINAL

#Poda
HIJOS = [x for x in HIJOS if x['ci'] < CotaSup ]

#Añadimos los hijos
NODOS.extend(HIJOS)

#Eliminamos el nodo ramificado
NODOS = [ x for x in NODOS if x['s'] != nodo_prometedor ]

print("La solucion final es:" ,MEJOR_SOLUCION , " en " , iteracion , "
↳iteraciones" , " para dimension: " ,DIMENSION )

ramificacion_y_poda(COSTES)

```

La solucion final es: [{'s': (1, 2, 0, 3), 'ci': 64}] en 10 iteraciones para dimension: 4

1.3 Generar matrices con valores aleatorios de mayores dimensiones (5,6,7,...) y ejecutar ambos algoritmos.

```

[ ]: import itertools
import numpy as np

# --- 1. CONTADOR DE OPERACIONES PARA FUERZA BRUTA ---
def fuerza_bruta_conteo(COSTES):
    n = len(COSTES)
    mejor_coste = float('inf')
    operaciones_fb = 0

    for p in itertools.permutations(range(n)):

```

```

operaciones_fb += 1 # Cuenta cada permutación evaluada
coste_actual = sum(COSTES[p[i]][i] for i in range(n))
if coste_actual < mejor_coste:
    mejor_coste = coste_actual

return operaciones_fb

# --- 2. ADAPTACIÓN DE TU ALGORITMO PARA CONTAR OPERACIONES ---
def ramificacion_y_poda_conteo(COSTES):
    DIMENSION = len(COSTES)
    MEJOR_SOLUCION = tuple(i for i in range(len(COSTES)))
    CotaSup = valor(MEJOR_SOLUCION, COSTES)

    NODOS = [{'s': ()}, {'ci': CI((), COSTES)}]
    operaciones_rp = 0

    while(len(NODOS) > 0):
        operaciones_rp += 1 # Cuenta cada ciclo de expansión/poda
        nodo_prometedor = [min(NODOS, key=lambda x: x['ci'])][0]['s']

        HIJOS = [{'s': x['s'], 'ci': CI(x['s'], COSTES)} for x in
↪ crear_hijos(nodo_prometedor, DIMENSION)]

        NODO_FINAL = [x for x in HIJOS if len(x['s']) == DIMENSION]
        if len(NODO_FINAL) > 0:
            if NODO_FINAL[0]['ci'] < CotaSup:
                CotaSup = NODO_FINAL[0]['ci']

        HIJOS = [x for x in HIJOS if x['ci'] < CotaSup]
        NODOS.extend(HIJOS)
        NODOS = [x for x in NODOS if x['s'] != nodo_prometedor]

    return operaciones_rp

# --- 3. TABLA COMPARATIVA DE OPERACIONES (DIMENSIONES 5, 6, 7, ...) ---
dimensiones = [4, 5, 6, 7, 8, 9]

print(f"{'Dimensión (n)':<15} | {'Operaciones Fuerza Bruta (n!)':<30} |_
↪ {'Operaciones Ramificación y Poda'}")
print("-" * 80)

for n in dimensiones:
    # Genera matrices de costes aleatorias
    matriz_aleatoria = np.random.randint(10, 100, size=(n, n)).tolist()

    # Operaciones de Fuerza Bruta
    ops_fb = fuerza_bruta_conteo(matriz_aleatoria)

```

```
# Operaciones de tu algoritmo de Ramificación y Poda
ops_rp = ramificacion_y_poda_conteo(matriz_aleatoria)

print(f"{n:<15} | {ops_fb:<30} | {ops_rp}")
```

Dimensión (n)	Operaciones Fuerza Bruta (n!)	Operaciones Ramificación y Poda
4	24	10
5	120	35
6	720	42
7	5040	1324
8	40320	176
9	362880	6450

1.4 ¿A partir de que dimensión el algoritmo por fuerza bruta deja de ser una opción?

Respuesta: Empíricamente realizando pruebas con el código el algoritmo por fuerza bruta es viable con dimension hasta $n=9$ que son 362.880 combinaciones pero cuando se realiza $n=10$ ya satura los recursos de procesamiento del sistema y el cálculo explota a $10! = 3.628.800$ operaciones.

1.5 ¿Hay algún valor de la dimensión a partir de la cual el algoritmo de ramificación y poda también deja de ser una opción válida?

Respuesta: el algoritmo de ramificación y poda también deja de ser viable a partir de dimensiones más avanzadas (típicamente cuando $n > 20$ o $n > 25$). Aunque la técnica de ramificación y poda (Branch and Bound) utiliza cotas inferiores y superiores para podar ramas enteras del árbol de decisiones y evitar evaluar todas las permutaciones posibles, el problema de asignación sigue siendo de complejidad NP-hard. Esto significa que, en el peor de los casos o cuando la estructura de los costes no permite realizar podas tempranas y efectivas, el espacio de búsqueda sigue creciendo de forma exponencial. Por lo tanto, al llegar a dimensiones grandes, la cantidad de nodos almacenados en memoria para evaluar las cotas (el arreglo NODOS) se vuelve masiva, lo que eventualmente agota la memoria del sistema y hace que el algoritmo también colapse de manera similar a la fuerza bruta, aunque con una tolerancia a la escala notablemente superior.

1.6 Descenso del gradiente

```
[ ]: import math #Funciones matematicas
import matplotlib.pyplot as plt #Generacion de gráficos (otra opcion seaborn)
import numpy as np #Tratamiento matriz N-dimensionales y otras
    ↪ (fundamental!)
import scipy as sc
from sympy import symbols, sin as sp_sin, cos as sp_cos, exp as sp_exp, diff
import random
```


Vamos a buscar el minimo de la funcion paraboloides :

$$f(x) = x^2 + y^2$$

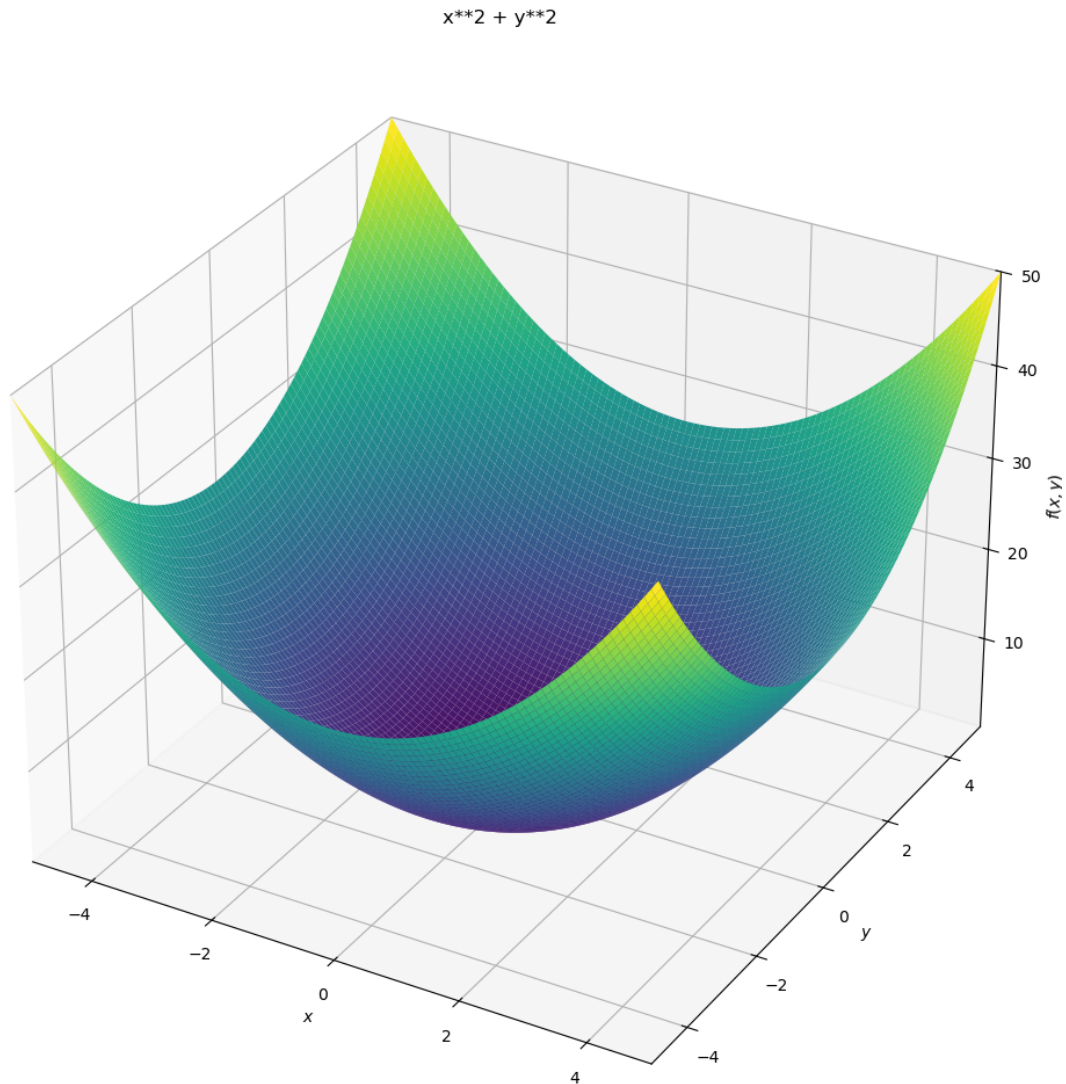
Obviamente se encuentra en (x,y)=(0,0) pero probaremos como llegamos a él a través del descenso del gradiente.

```
[ ]: #Definimos la funcion
#Paraboloides
f = lambda X: X[0]**2 + X[1]**2 #Funcion
df = lambda X: [2*X[0] , 2*X[1]] #Gradiente

df([1,2])
```

```
[ ]: [2, 4]
```

```
[ ]: from sympy import symbols
from sympy.plotting import plot
from sympy.plotting import plot3d
x,y = symbols('x y')
plot3d(x**2 + y**2,
      (x,-5,5),(y,-5,5),
      title='x**2 + y**2',
      size=(10,10))
```



```
[ ]: <sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at 0x216d4f32a50>
```

```
[ ]: #Prepara los datos para dibujar mapa de niveles de Z  
resolution = 100  
rango=5.5  
  
X=np.linspace(-rango,rango,resolucion)  
Y=np.linspace(-rango,rango,resolucion)  
Z=np.zeros((resolucion,resolucion))  
for ix,x in enumerate(X):  
    for iy,y in enumerate(Y):
```

```

Z[iy,ix] = f([x,y])

#Pinta el mapa de niveles de Z
plt.contourf(X,Y,Z,resolucion)
plt.colorbar()

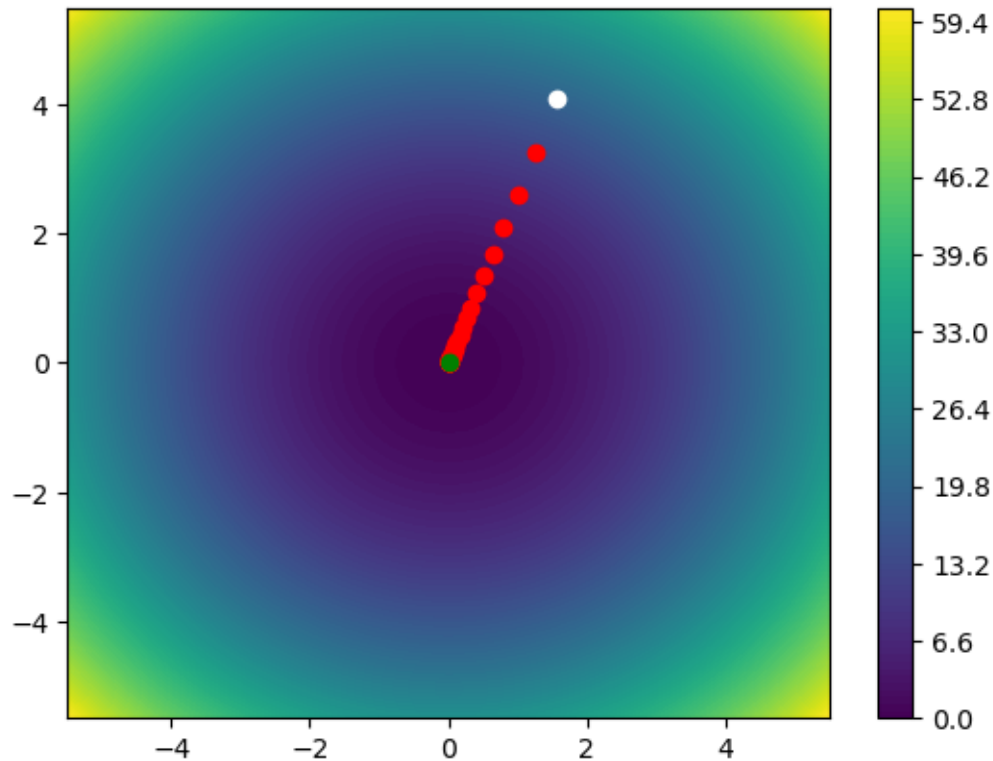
#Generamos un punto aleatorio inicial y pintamos de blanco
P=[random.uniform(-5,5 ),random.uniform(-5,5 ) ]
plt.plot(P[0],P[1],"o",c="white")

#Tasa de aprendizaje. Fija. Sería más efectivo reducirlo a medida que nos
↪acercamos.
TA=.1

#Iteraciones:50
for _ in range(50):
    grad = df(P)
    #print(P,grad)
    P[0],P[1] = P[0] - TA*grad[0] , P[1] - TA*grad[1]
    plt.plot(P[0],P[1],"o",c="red")

#Dibujamos el punto final y pintamos de verde
plt.plot(P[0],P[1],"o",c="green")
plt.show()
print("Solucion:" , P , f(P))

```



Solucion: [2.2231757041672776e-05, 5.8319197039452966e-05]
3.8953797644865065e-09

¿Te atreves a optimizar la función?:

$$f(x) = \sin\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) * \cos(2x + 1 - e^y)$$

Derivada:

$$\frac{\partial f}{\partial x} = x \cos\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \cos(2x + 1 - e^y) - 2 \sin\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \sin(2x + 1 - e^y)$$

```
[ ]: #Definimos la funcion
f= lambda X: math.sin(1/2 * X[0]**2 - 1/4 * X[1]**2 + 3) *math.cos(2*X[0] + 1 -
↪math.exp(X[1]))
# Definimos el gradiente usando las derivadas parciales manuales
df = lambda X: [
    # Derivada respecto a x (df/dx)
    X[0] * math.cos(0.5 * X[0]**2 - 0.25 * X[1]**2 + 3) * math.cos(2 * X[0] + 1 -
↪math.exp(X[1])) - 2 * math.sin(0.5 * X[0]**2 - 0.25 * X[1]**2 + 3) * math.
↪sin(2 * X[0] + 1 - math.exp(X[1])),

    # Derivada respecto a y (df/dy)
```

```

-0.5 * X[1] * math.sin(0.5 * X[0]**2 - 0.25 * X[1]**2 + 3) * math.cos(2 *
↪X[0] + 1 - math.exp(X[1])) + math.exp(X[1]) * math.sin(0.5 * X[0]**2 - 0.25
↪* X[1]**2 + 3) * math.sin(2 * X[0] + 1 - math.exp(X[1]))
]

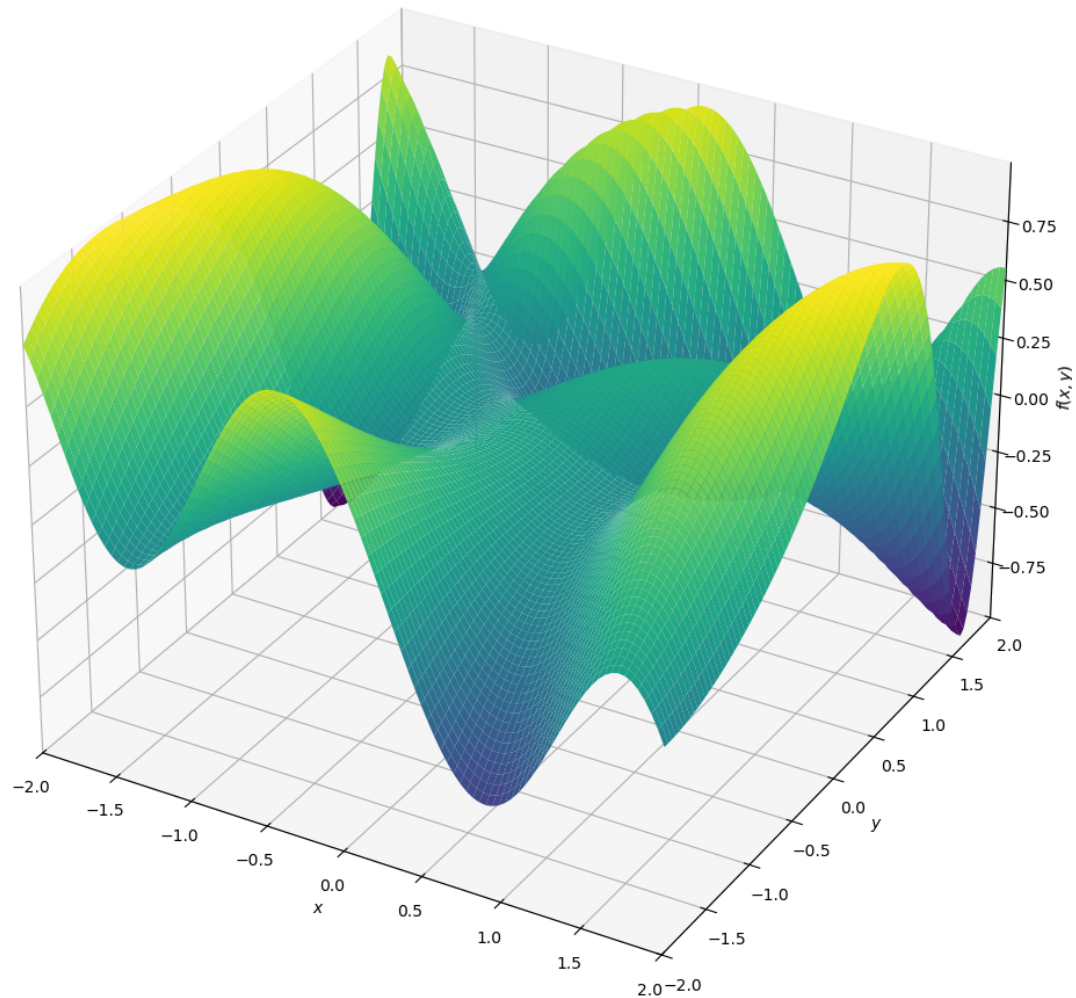
```

```

[ ]: from sympy import symbols, sin, cos, exp
from sympy.plotting import plot
from sympy.plotting import plot3d
x,y = symbols('x y')
f_sym = sin(0.5 * x**2 - 0.25 * y**2 + 3) * cos(2 * x + 1 - exp(y))
plot3d(f_sym,
      (x,-2,2),(y,-2,2),
      title='Superficie 3D: f(x, y) = sin(0.5*x^2 - 0.25*y^2 + 3) * cos(2*x +
↪1 - exp(y))',
      size=(10,10))

```

Superficie 3D: $f(x, y) = \sin(0.5x^2 - 0.25y^2 + 3) * \cos(2x + 1 - \exp(y))$



```
[ ]: <sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at 0x216d60e25d0>
```

```
[ ]: # 3. Preparamos los datos para el mapa de niveles (Contorno 2D)
    resolution = 100
    rango = 3.0

    X_vals = np.linspace(-rango, rango, resolution)
    Y_vals = np.linspace(-rango, rango, resolution)
    Z = np.zeros((resolution, resolution))

    for ix, xv in enumerate(X_vals):
```

```

    for iy, yv in enumerate(Y_vals):
        Z[iy, ix] = f([xv, yv])

# Pintamos el mapa de contorno
plt.figure(figsize=(8, 6))
plt.contourf(X_vals, Y_vals, Z, resolution, cmap='viridis')
plt.colorbar(label='Valor de f(x, y)')

# 4. Generamos un punto aleatorio inicial y lo pintamos de blanco
P = [random.uniform(-1.5, 1.5), random.uniform(-1.5, 1.5)]
plt.plot(P[0], P[1], "o", c="white", markersize=8, label="Punto Inicial")

# 5. Parámetros del Descenso del Gradiente
TA = 0.01 # Tasa de aprendizaje pequeña debido a las oscilaciones
iteraciones = 80

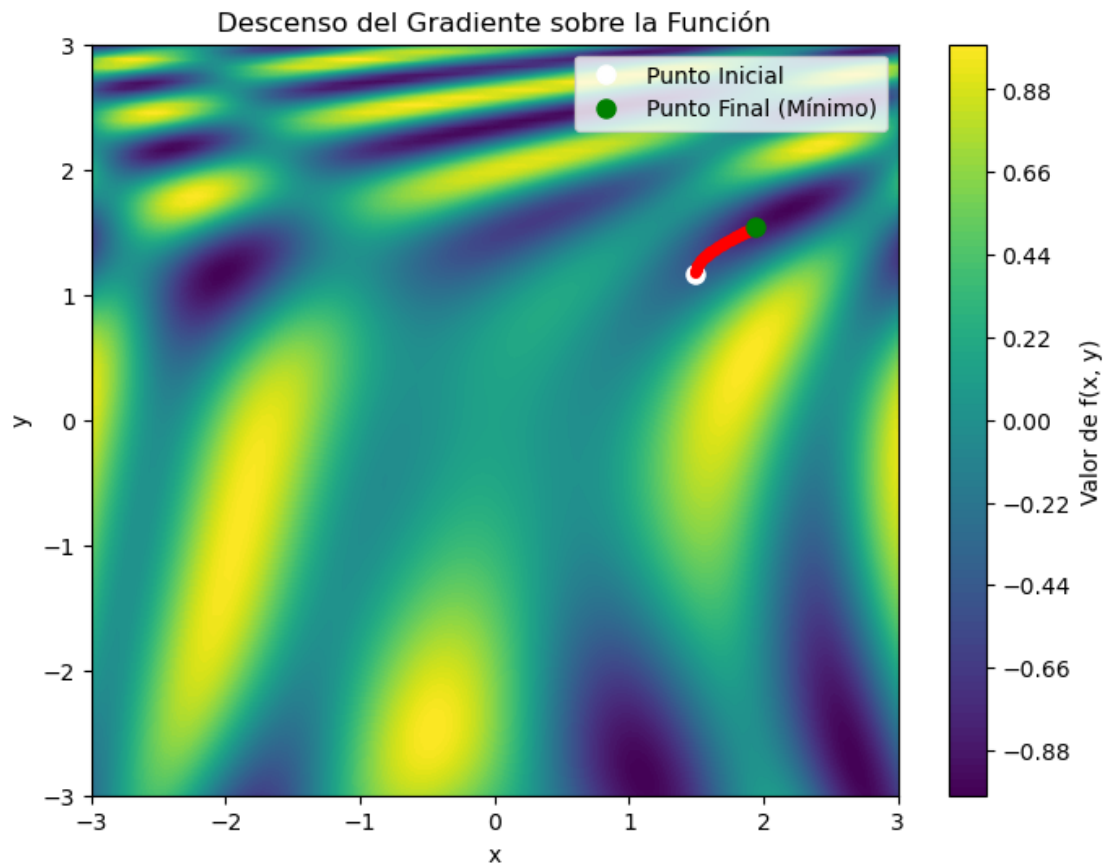
# Bucle de iteraciones (pintando los pasos en rojo)
for _ in range(iteraciones):
    grad = df(P)
    P[0] = P[0] - TA * grad[0]
    P[1] = P[1] - TA * grad[1]
    plt.plot(P[0], P[1], "o", c="red", markersize=4)

# 6. Pintamos el punto final de verde
plt.plot(P[0], P[1], "o", c="green", markersize=8, label="Punto Final (Mínimo)")

plt.title("Descenso del Gradiente sobre la Función")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()

print("Solución final optimizada (x, y):", P)
print("Valor mínimo alcanzado por la función:", f(P))

```



Solución final optimizada (x, y): [1.9355529789417436, 1.5389268556372342]

Valor mínimo alcanzado por la función: -0.8881852505964705